進階程式設計
Advanced Program Design

L5

Chia-Chi Chang
ccchang@cc.ncue.edu.tw

程式/演算法效能評估

- 評估程式(演算法)的效率通常用以下的方式來判斷：
  - 時間複雜度(Time Complexity)：能用最短時間去達成目的。
    ※排除編譯器優化、CPU處理器、執行時脈等因素
  - 空間複雜度(Space Complexity)：能用最少的記憶體空間去達成目的。
  - 時間複雜度常用於評斷程式/演算法是否最佳。
- 時間複雜度應從執行時間=執行次數x每一行程式執行時間，化簡成

  執行時間 ∝ 執行次數 (每行程式執行時間假設為常數)
- 不同的輸入條件下，會有不同的執行結果：
  - 最糟情況(Worst Case)：需要最長時間才能完成結果
  - 最佳情況(Best Case)：需要最短時間(※輸入即等於輸出)就能完成結果。

O(n), Big-O of n和Ω(n), Omega of n

- Big-O常被用來表示該程式/演算法的上限(Upper Bound)（時間複雜度最糟的狀況）

定義：$f(n) = O(g(n))$，若且唯若存在一個正整數c和n_0，當$n \geq n0$時，$f(n) \leq c*g(n)$。

Q1：假設n個元素的陣列arr，若$f(n) = 3n+2$，證明$O(f(n)) = O(n)$

$f(n) \leq c*g(n)$

$\therefore 3n + 2 \leq c*g(n)$

當$g(n) = n$時

$3n + 2 \leq c*g(n)$ 變成$2 \leq (c-3)n$

若$c = 4$且$n_0 \geq 2$時，則$O(3n + 2)$可以表示成$O(n)$

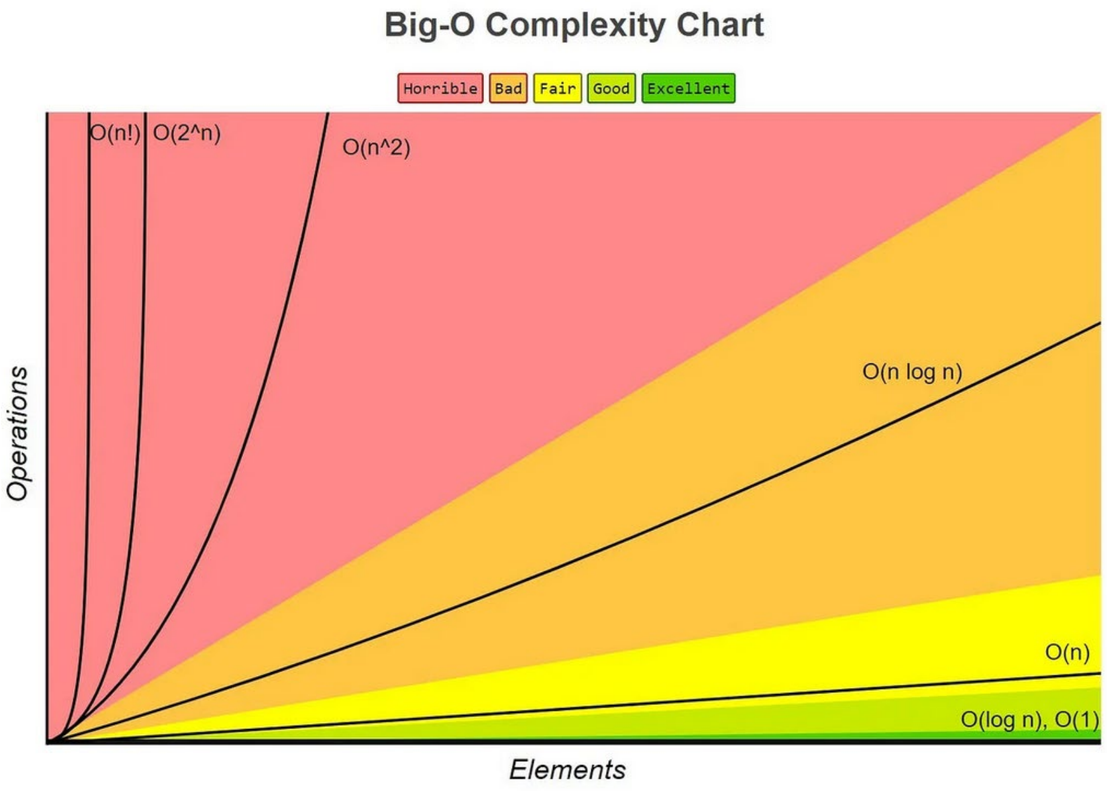
- Big-O取「執行次數」中最高次方(最大指數)

- Ω(n)常被用來表示該程式/演算法的下限(Lower Bound)（時間複雜度最好的狀況）

定義：$f(n) = \Omega(g(n))$，若且唯若，存在一個正整數c及n_0，當$n \geq n_0$時，$f(n) \geq c*g(n)$

時間複雜度等級

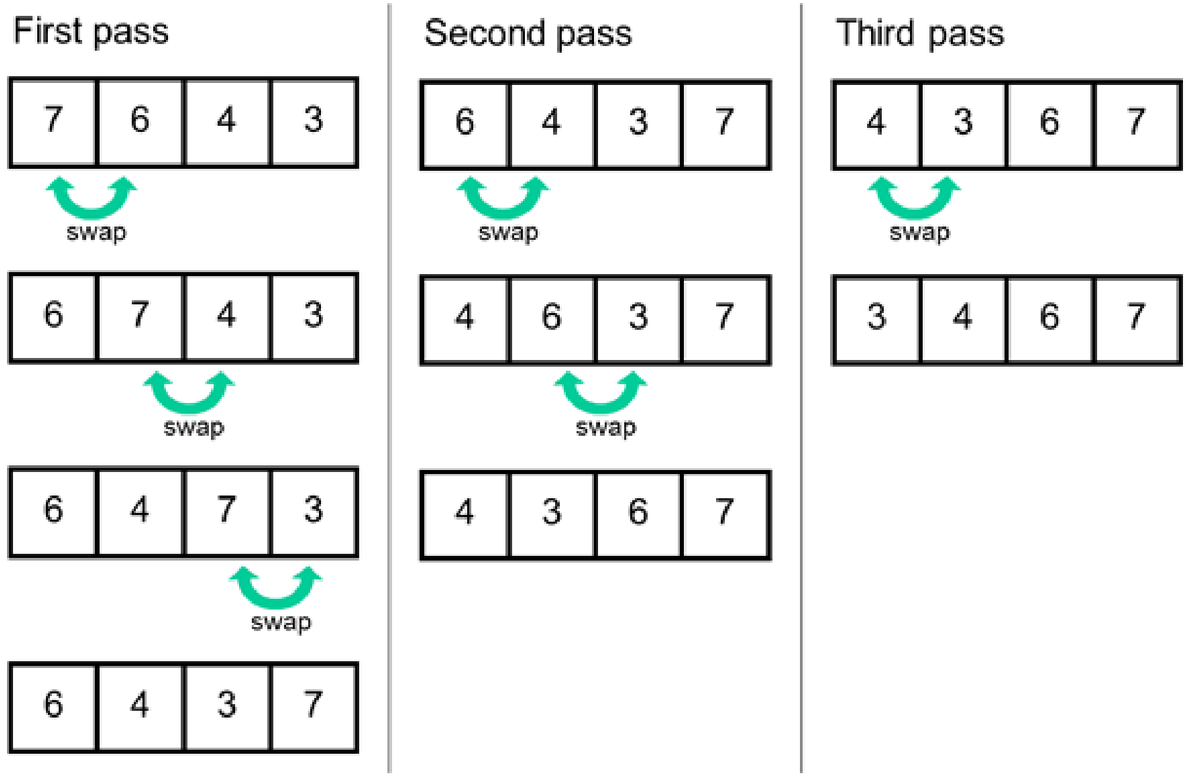
執行時間	等級
短	O(1)：常數時間(Constant Time)
↑	O(logn)：對數時間
	O(n)：線性時間
	O(nlogn)：線性乘對數時間
	O(n^2)：平方時間
	O(n^3)：立方時間
	O(2^n)：指數時間
長	O(n!)：階乘時間



Big-O Complexity Chart: http://bigocheatsheet.com/

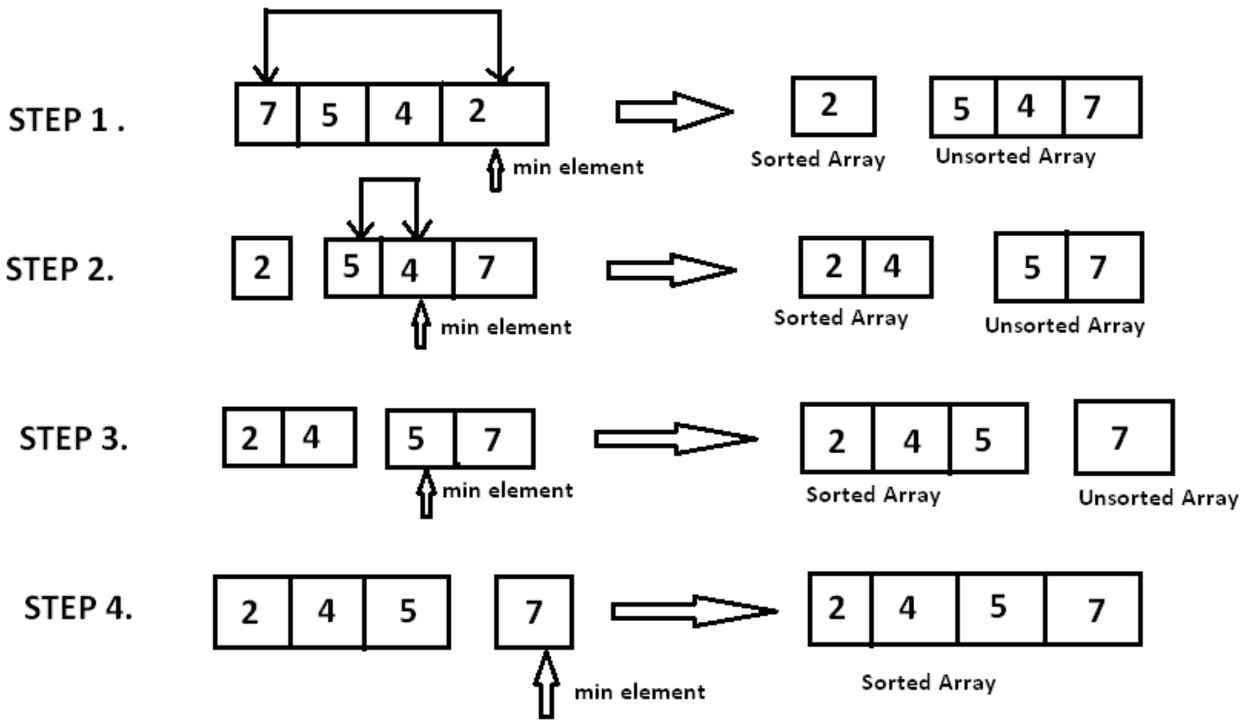
氣泡排序法(Bubble Sorting)[$O(n^2)$]

- 若要進行由小排至大，須將相鄰兩個數值進行比較，將「數字大」交換至右邊，如下圖所示；反之，若由大排至小，則須將「數字小」交換至右邊。
  - 若有n個資料要排序，通常需要n-1輪掃描；每進行完一輪排序數目會遞減1(※因為每一輪的最右邊的數值為該輪掃描中的最大者)。
  - Worst Case：$O(n^2)$，Best Case：O(n)「至少要比較n次」。
- 共執行(n-1)+(n-2)+…+2+1=n(n-1)/2次，時間複雜度$O(n^2)$



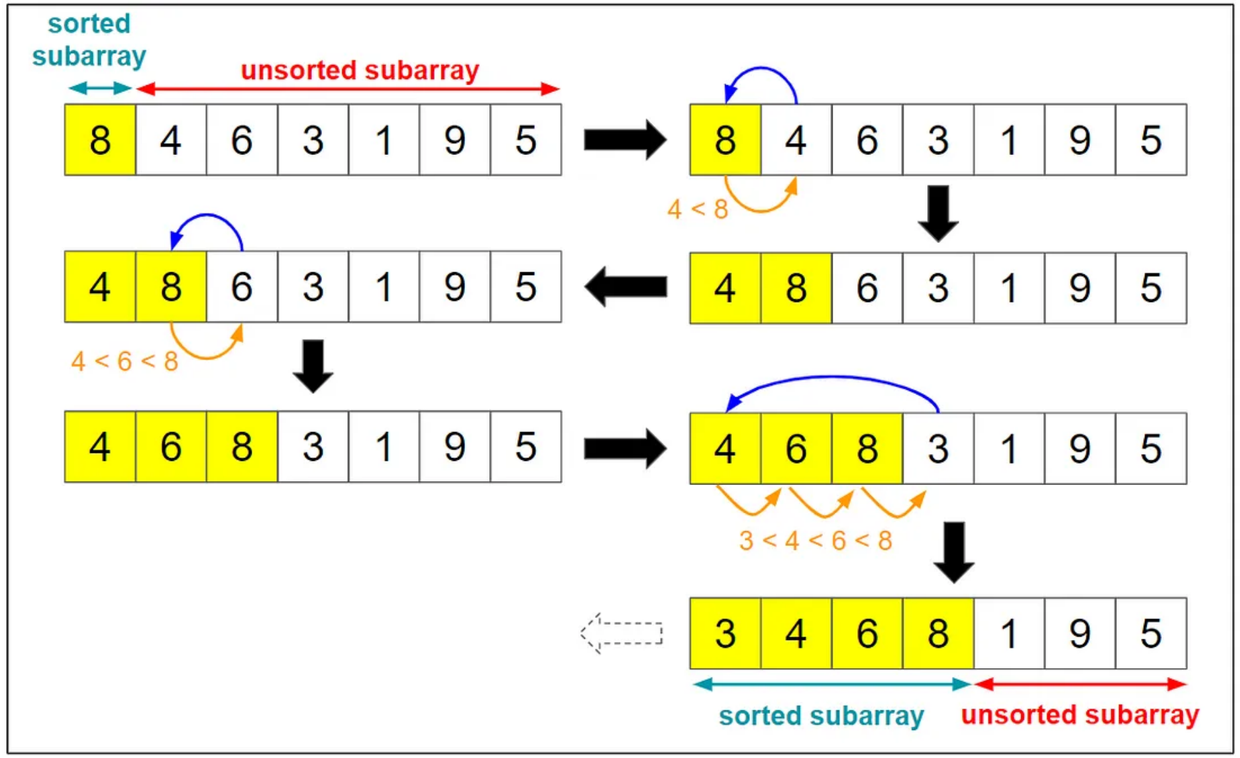
選擇排序法(Selection Sorting)[$O(n^2)$]

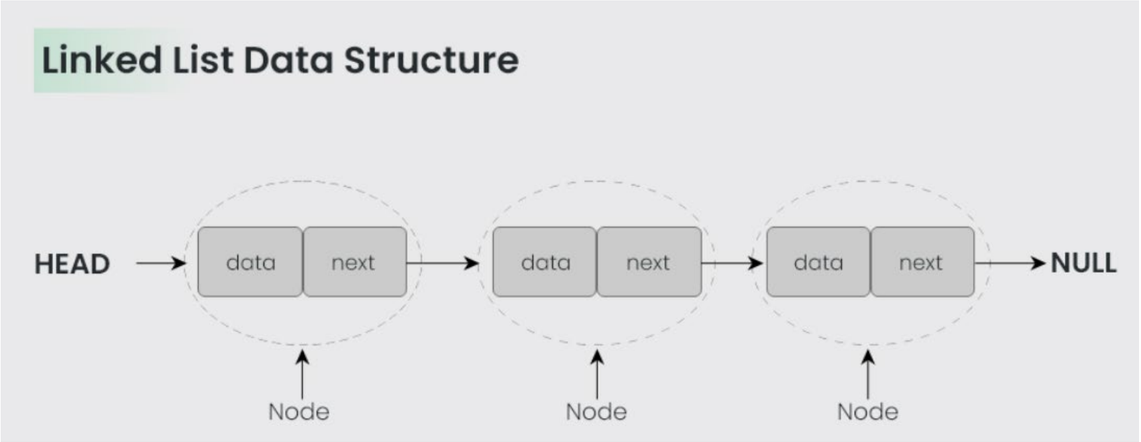
- 若要進行由小至大，每一輪找出最小者並記住其索引值，然後再進行資料交換(不同於氣泡排序法，只須交換1次)，依此循環至最後1個數值。
- Worst Case：$O(n^2)$，Best Case：$O(n^2)$
- 比較次數不受輸入資料影響，均為n(n-1)/2



插入排序法(Insertion Sorting)[O(n^2)]

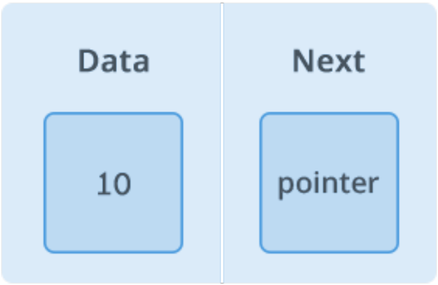
- 將資料列假設分成已排序和未排序的兩部分，每次從未排序的資料中，挑選出一個元素，插入到已排序的資料中，直到所有的資料都已排序完成。
- Worst Case：n(n-1)/2，Best Case：n-1
- 當插入於黃色區塊的數字是最小時，黃色區塊內的資料需要進行大量搬移，因此鏈結串列(Linked List)是適當的資料存取方式。
  - 缺點：鏈結串列需要2倍記憶體空間（每個node需要1個空間存放指標next）



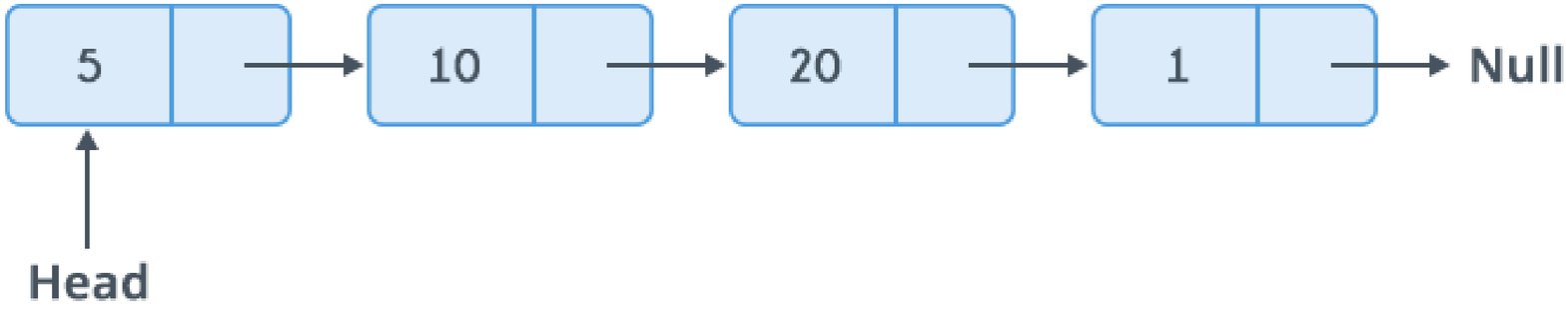


鏈結串列(Linked List)

- 節點(Node)：由資料Data和指標Pointer組合而成（C/C++語言有指標）

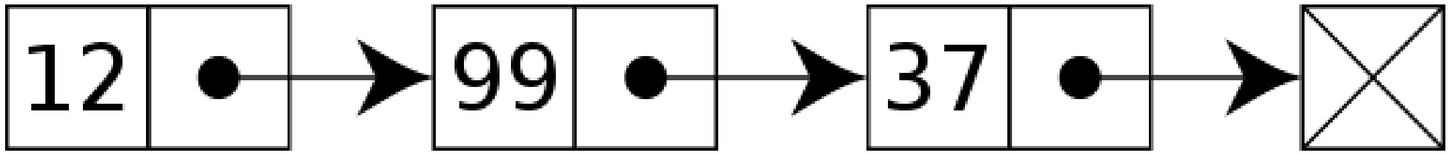


- 每個節點的Next指標會「指向」下個節點，亦即Next指標存在下個節點的位址。
- Head指標指向第1個節點的位址，最後1個節點的Next指標指向Null(空位址)。
- 走訪節點：根據指標指示去逐一找尋某個節點，直到指向Null。



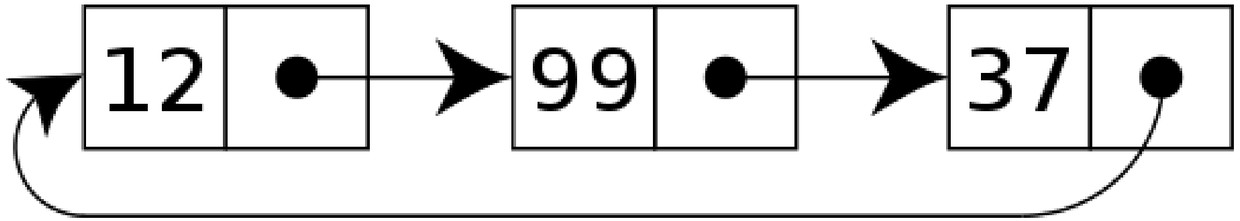
常見的鏈結串列架構

1. 單向鏈結串列(Singly Linked List)

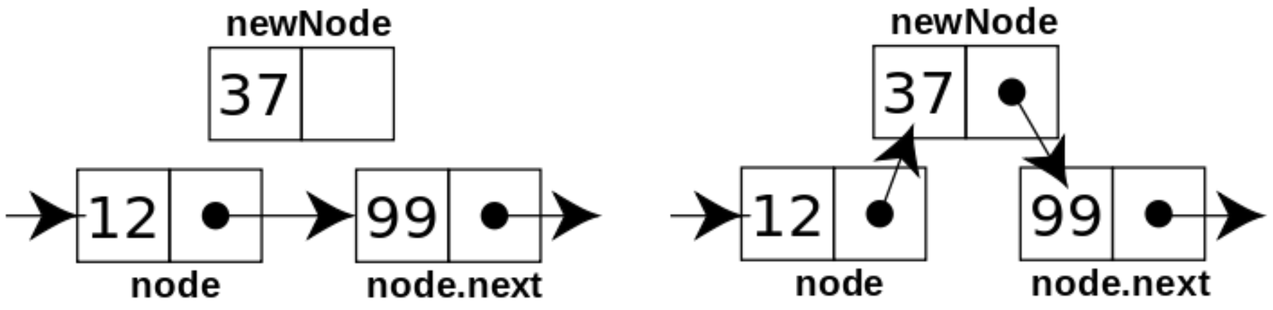


2. 雙向鏈結串列(Doubly Linked List)

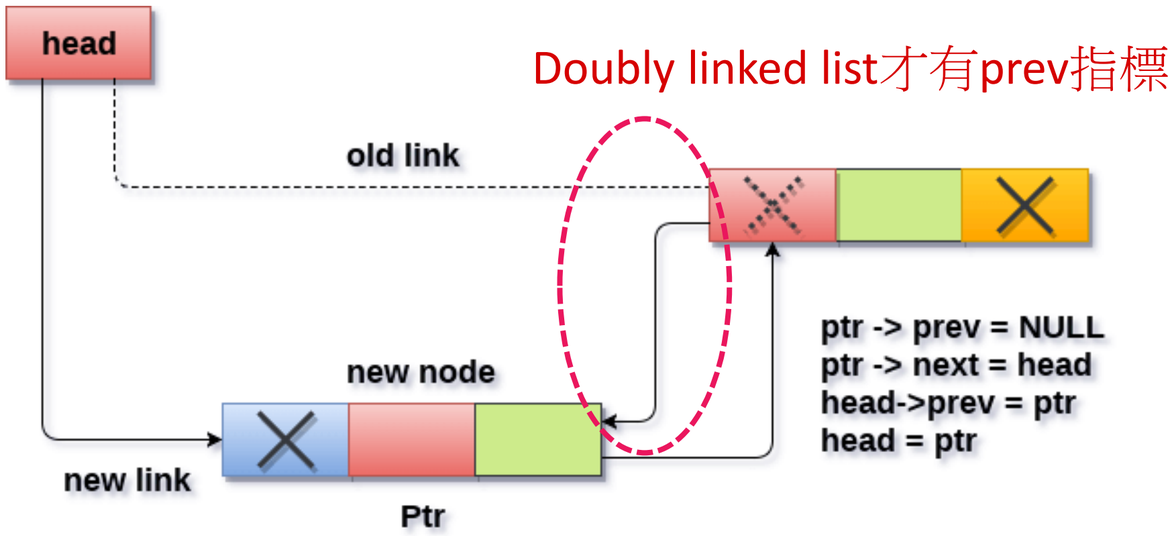
3. 環狀串列(Circularly Linked List)



Insert a node after an existing node in a linked list

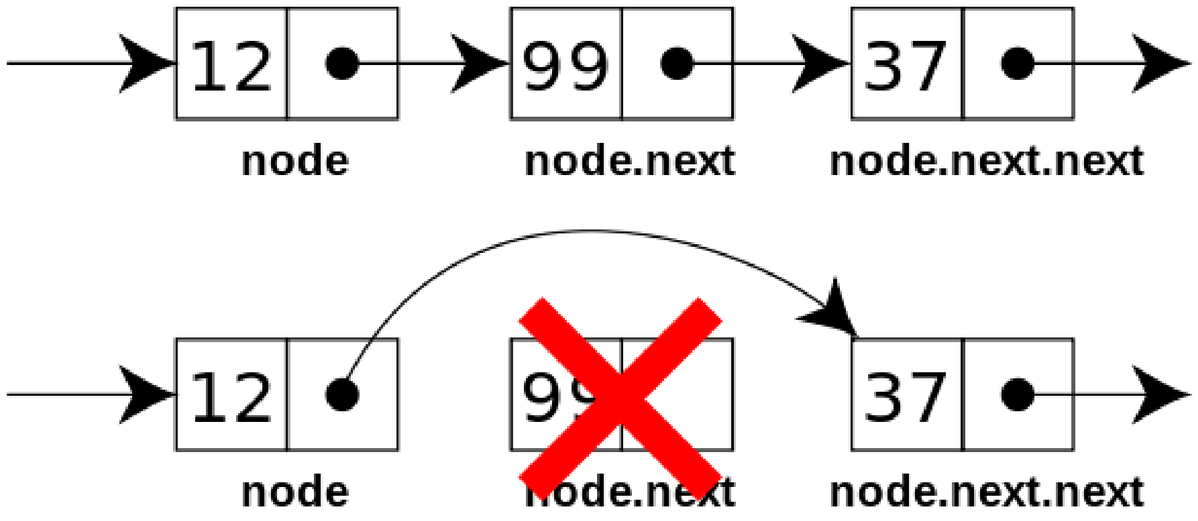


Singly linked list



Insertion into doubly linked list at beginning

Delete a node from a linked list



鏈結串列的延伸-動態記憶體配置malloc()和calloc()

- 鏈結串列會經常新增和刪除節點，動態記憶體配置(Dynamic Memory Allocation)可以有效利用記憶體空間。

型別	說明
size_t	size_t型別定義在stddef.h， ▪ 32-bit環境為unsigned int(4 Bytes)【4294967295位元組】 ▪ 64-bit環境為unsigned long(8 Bytes)【18446744073709551615位元組】

函式	說明
void *malloc(size_t size)	定義1個連續記憶體空間，參數size為記憶體區塊，單位為位元組 ※記憶體空間內容「未初始化」
void *calloc(size_t count, size_t size)	定義1個連續記憶體空間，尺寸為count x size，可視為1個陣列內有count元素，每個元素佔有size記憶體區塊，單位為位元組 ※記憶體空間內容「均初始化為0」

動態產生新的記憶體區塊：malloc()和calloc()

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 int main(int argc, char *argv[]) {
04     int loop;
05     int *ptriArr = (int*)malloc(5*sizeof(int));
06     float *ptrfArr = (float*)calloc(5,sizeof(int));
07     //Check if the memory has been successfully allocated by
08     //malloc or calloc or not
09     if(ptriArr == NULL || ptrfArr == NULL) {
10         printf("!\n");
11     } else {
12         for(loop=0; loop<5; loop++) {
13             ptriArr[loop] = 100 + loop;
14             *(ptrfArr + loop) = 10 + 0.5 * (float)loop;
15         }
16         for(loop=0; loop<5; loop++) {
17             printf("ptriArr[%d] = %d, ",loop,ptriArr[loop]);
18             printf("ptrfArr[%d] =
%.1f\n",loop,*(ptrfArr+loop));
19         }
20     }
21     free(ptriArr);  //Malloc memory has been freed.
22     free(ptrfArr);  //Calloc memory has been freed.
23     system("PAUSE");
24     return 0;
25 }
```

//配置5個存放int記憶體區塊(5*4 Bytes)

//配置5個存放float記憶體區塊(5*4 Bytes)

//※檢查ptriArr和ptrfArr不是NULL(空指標)，代表系統成功配置動態記憶體空間

```
ptriArr[0] = 100, ptrfArr[0] = 10.0
ptriArr[1] = 101, ptrfArr[1] = 10.5
ptriArr[2] = 102, ptrfArr[2] = 11.0
ptriArr[3] = 103, ptrfArr[3] = 11.5
ptriArr[4] = 104, ptrfArr[4] = 12.0
Press any key to continue . . .
```

//※須手動釋放動態記憶體空間

釋放配置記憶體：free()

- 動態記憶體配置會從「配置(呼叫malloc(), calloc())」開始，到「釋放(呼叫free())」結束，或者到程式結束。
- 若無使用的動態記憶體空間，使用者須自行歸還給系統，否則會造成記憶體洩漏(memory leakage)。
- fp可以是malloc()、calloc()或realloc()回傳的記憶體空間指標或是空指標，對於已經釋放過的動態記憶體位址或其他值，程式行為沒有定義。
- 由於free()沒有返回值，也無法得知是否成功釋放動態記憶體空間。

函式	說明
void free(void *fp)	fp指標為指向動態記憶體的動態空間指標

重新配置記憶體：realloc()

- 若原有的動態記憶體空間不敷使用，可以使用realloc()增加或減少動態記憶體區塊的大小。
- 如同malloc()和calloc()，應檢查realloc()回傳的重新配置動態記憶體空間指標，若是NULL指標則表示無法重新配置所需的記憶體空間

函式	說明
void *realloc(void *fp, size_t size)	釋放fp指標指向的動態記憶體空間，並配置size位元組的記憶體區塊。 1. 若fp有之前舊動態記憶體位址，會保留原始記憶體內容 2. 若fp指向新的動態記憶體位址，那麼會將原始內容複製到新的記憶體區塊，新的記憶體區塊超過舊有資料的部份則未指定(※不應該假設內容為0) 3. 若fp為空指標，realloc()等同於malloc()，配置1個新的動態記憶體區塊 4. 若返回值不是fp，則新記憶體區塊的位址有可能不與舊記憶體區塊位置相同。(ex. nptr = realloc(ptr,3*sizeof(int);) ptr是指向舊記憶體區塊位址，nptr是指向新記憶體區塊位址，nptr不一定會跟ptr一樣。

The End